

Финальный проект курса LLM Engineer



Техническое задание

Дедлайн сдачи: 20 мая • Demo Days: 26-28 мая

1. Цель проекта

Финальный проект - это возможность применить все навыки, полученные на курсе, для создания законченного LLM-продукта, решающего реальную задачу. Проект должен закрывать конкретную боль (рабочую, бытовую или бизнесовую) либо представлять собой стартап-идею, готовую к презентации инвесторам или первым пользователям.

Задача - не «собрать демо», а спроектировать и реализовать продукт инженерного уровня: с архитектурой, мониторингом и осознанным выбором каждого технического решения. Менторы при оценке смотрят прежде всего на ваше мышление и обоснование, а не на красоту интерфейса.

2. Формат работы

- Индивидуально или в команде до 3 человек.
- Тема - свободная. Список рекомендованных направлений приведён в разделе 8.
- Платформа реализации - веб- или мобильное приложение (frontend обязателен, не CLI).
- Защита проекта - публичная, на Demo Days, продолжительностью до 10 минут.

3. Обязательные технические требования



Все пункты ниже являются критическими. Проект, в котором отсутствует хотя бы один из обязательных модулей, к защите не допускается.

3.1. Архитектура и оркестрация

- Оркестрация агентов через LangGraph, CrewAI или Parlant. Обязательно наличие как минимум одного многошагового workflow с условной логикой (ветвления, циклы, человеческое подтверждение).
- Минимум один собственный MCP-сервер, разработанный участником и интегрированный в проект. Сервер должен предоставлять как минимум 2-3 содержательных tool'a.
- Минимум один собственный Skill (например, для Claude или аналогичной системы), оформленный по стандартам - с [SKILL.md](#), описанием триггеров и валидной структурой.

3.2. Работа с данными

- RAG-пайплайн с осознанным выбором: chunking-стратегия, embedding-модель, векторная БД (Qdrant / Chroma / Pinecone / pgvector), reranker (опционально, но приветствуется).
- Обработка документов или веб-сайтов: парсинг PDF / DOCX / HTML, либо скрапинг сайтов с обработкой динамического контента.
- Мультимодальность: проект обязан использовать как минимум одну мультимодальную возможность - анализ изображений, OCR, обработка аудио (speech-to-text), анализ видео или генерация изображений. Использование должно быть осмысленным, а не «прикручено для галочки».

3.3. Мониторинг и оценка

- Логирование и трейсинг всех LLM-вызовов через LangSmith (или аналог - Langfuse, Weights & Biases). На защите показать дашборд с реальными трейсами.

- Evals: golden dataset не менее 30 примеров, автоматизированный прогон оценок, как минимум 2 метрики (например, accuracy + LLM-as-judge / faithfulness / relevance).
- A/B тестирование: как минимум один эксперимент со сравнением двух конфигураций (разные модели, промпты, температуры, RAG-стратегии) с метриками и выводами.

3.4. Гиперпараметры и оптимизация

- Документированный выбор LLM с обоснованием: почему GPT / Claude / Gemini / локальная модель. Баланс стоимости, латентности и качества должен быть описан явно.
- Подбор температуры, top_p, max_tokens - с экспериментальным обоснованием.

4. Рекомендуемые требования (влияют на оценку)



Эти пункты не обязательны для допуска к защите, но напрямую повышают итоговую оценку и качество проекта. Чем больше реализовано - тем сильнее проект.

- Guardrails: входная и выходная фильтрация - защита от prompt injection, PII-фильтрация, контроль токсичности или доменных ограничений. Реализация через Guardrails AI, NeMo Guardrails или собственная.
- Кэширование (semantic cache, prompt cache) для снижения стоимости.
- Fallback-стратегия между моделями (если основная недоступна или превышен бюджет).
- Контейнеризация через Docker и docker-compose.
- CI/CD пайплайн (GitHub Actions) с автозапуском evals на каждый PR.

- Деплой на публичный URL (Vercel, Railway, Render, Fly.io, AWS).
 - Аутентификация пользователей и базовая роль-модель.
 - Голосовой интерфейс (audio-to-audio модели).
 - Fine-tuning или LoRA-адаптация модели под доменную задачу.
 - Собственный eval-фреймворк или нестандартные метрики.
 - Интеграция с реальными внешними API (банки, CRM, мессенджеры).
 - Реальные пользователи и собранная обратная связь.
-

5. Артефакты сдачи

К дедлайну 20 мая команда должна предоставить:

- Публичный или приватный (с доступом для менторов) GitHub-репозиторий с полным кодом проекта.
 - md с описанием проекта, архитектурой, инструкцией по запуску, скриншотами и ссылкой на демо.
 - Файл ARCHITECTURE.md или mindmap-диаграмма архитектуры (Excalidraw, Miro, draw.io).
 - Файл EVALS.md с golden dataset, метриками и результатами A/B экспериментов.
 - Презентация защиты (10-15 слайдов, PDF или Google Slides)..
-

6. Формат защиты и критерии оценки

6.1. Регламент защиты

- Длительность выступления - не более 10 минут.
- Структура: проблема → решение → демо → архитектура → метрики и evals → выводы.
- После выступления - Q&A с менторами (5-7 минут).

- Менторы оценивают прежде всего ваше мышление: как вы пришли к решению, какие альтернативы рассматривали, что измеряли и почему.

6.2. Критерии оценки и контрольные вопросы



Каждый критерий оценки сопровождается набором вопросов, которые менторы могут задать на защите. Эти вопросы - не формальность, а главный инструмент оценки. Готовьтесь не к ответу «что вы сделали», а к ответу «почему именно так, а не иначе».

Критерий	Вес	Вопросы, через которые проверяется
Архитектура и техническое решение	40%	• Почему именно LangGraph (или CrewAI / Parlant)? Что вы рассматривали как альтернативу и почему отказались? • Как устроен поток управления в вашем графе? Где есть ветвления, циклы, human-in-the-loop? • Покажите архитектурную диаграмму и проведите по пути одного запроса от пользователя до ответа. • Какие компоненты системы независимы и могут быть заменены? Где вы сознательно выбрали coupling и почему? • Почему выбрана именно эта LLM (GPT / Claude / Gemini / локальная)? Как сбалансированы стоимость, скорость и качество? • Сколько стоит один пользовательский запрос в среднем? Что вы делали для оптимизации стоимости? • Что произойдёт, если основная LLM недоступна или резко подорожает? Какая стратегия? • Какие trade-off вы осознанно приняли (например, качество против latency, простота против гибкости)? • Какие у вас были гипотезы на старте,

Критерий	Вес	Вопросы, через которые проверяется
		которые не подтвердились? Что вы из этого узнали?
Качество LLM-инженерии	25%	• Как эволюционировал ваш главный промпт? Покажите v1 и текущую версию - что и почему изменилось? • Что в golden dataset и почему именно эти 10+ примеров? Как они покрывают edge cases? • Какие метрики вы измеряете и почему именно их? Что эти метрики НЕ показывают? • Расскажите про ваш A/B эксперимент: гипотеза, что сравнивали, результат, какое решение приняли? • Как устроен ваш RAG: chunking, embeddings, retrieval, reranking? Какие альтернативы пробовали? • Какие edge cases ломают систему сейчас? Как вы это обнаружили?
Полнота покрытия обязательных модулей	20%	• Покажите ваш MCP-сервер. Почему именно эти tool'ы и почему MCP, а не обычный API? • Покажите Skill: что в <u>SKILL.md</u>, как срабатывают триггеры, в каком сценарии Skill полезнее обычного промпта? • Где и почему в проекте используется мультимодальность? Что было бы потеряно без неё? • Покажите дашборд LangSmith с реальными трейсами одного пользовательского сценария.
Презентация и защита	10%	• Можете ли вы за 60 секунд объяснить суть проекта человеку, не знакомому с LLM? • Структура выступления: понятна ли история «проблема → решение → результат»? • Качество архитектурной диаграммы (mindmap): отражает ли она реальное

Критерий	Вес	Вопросы, через которые проверяется
		устройство системы? • Уверенность и ясность ответов на Q&A: команда понимает свой код или просто его написала?
Бизнес-ценность и решение реальной проблемы	5%	• Кто конкретный пользователь вашего продукта? Опишите его одним предложением. • Какую боль вы закрываете и как пользователь решает её сейчас, без вашего продукта? • Пробовали ли вы дать продукт реальным людям? Что они сказали? • Что должно произойти, чтобы вы продолжили развивать этот проект после курса?

+ Реализованные рекомендуемые требования из раздела 4 могут добавить до +10% сверху, но итоговая оценка не превышает 100%.

7. Рекомендованные направления тем

Тема свободная, но ниже - направления, которые хорошо ложатся на обязательные требования. Можно брать как есть, адаптировать или предложить своё.

Рабочая автоматизация

- Анализатор и скрининг резюме: парсинг PDF, мультимодальность для сканов, RAG по требованиям вакансии.
- Генератор документации на основе pull-request'ов: MCP с GitHub, RAG по кодстайлу команды.
- Code review агент: MCP с GitLab/GitHub, evals на исторических PR.

- AI-ассистент для подготовки к техническим собеседованиям с голосовым mock-интервью.

Мониторинг и аналитика

- AI growth manager: скрапинг Reddit / X / YouTube / LinkedIn, анализ виральности, еженедельный playbook.
- Агент для рынка недвижимости (Krisha.kz, OLX): динамика цен, детект недооценённых объектов, инвест-рекомендации.
- Мониторинг репутации бренда: трекинг упоминаний, sentiment-анализ, алерты.
- Финансовый аналитик: PDF-выписки банков, скрапинг курсов, прогнозы, MCP для Kaspi/Halyk.

Персональные ассистенты

- Фитнес-агент в Telegram: персональные программы, мультимодальность (фото прогресса), работа с оттоком.
- Юридический ассистент для МСБ в Казахстане: RAG по законодательству РК, обработка договоров, MCP с egov.kz.
- Медицинский триаж-агент: фото симптомов, RAG по протоколам, эскалация к врачу.
- Personal knowledge manager: обработка заметок, веб-страниц, голосовых, RAG поверх личной базы.

Бизнес-инструменты

- AI-агент для оптимизации работы ресторана / фитнес-клуба.
- Извлечение расходов из банковских выписок с автокатегоризацией.
- Контент-агент для маркетологов: анализ трендов, генерация сценариев, мультимодальный анализ thumbnail'ов.

8. Финальный чек-лист самопроверки

Перед сдачей проверьте, что у вас есть всё перечисленное:

- ☐ Собственный MCP-сервер с 2+ tool'ами
- ☐ Собственный Skill с SKILL.md
- ☐ LangGraph / CrewAI / Parlant с многошаговым workflow
- ☐ RAG-пайплайн с обоснованным выбором компонентов
- ☐ Обработка документов или скрапинг сайтов
- ☐ Мультимодальность (vision / OCR / audio / видео)
- ☐ LangSmith логирование с реальными трейсами
- ☐ Golden dataset 10+ примеров и автоматизированные evals
- ☐ A/B эксперимент с выводами
- ☐ Обоснованный выбор LLM и гиперпараметров
- ☐ Веб- или мобильный фронтенд
- ☐ GitHub-репозиторий с README и инструкцией запуска
- ☐ ARCHITECTURE.md или mindmap
- ☐ EVALS.md с метриками и результатами
- ☐ Презентация
- ☐ Развёрнутое демо или однокомандный локальный запуск



Удачи! Стройте то, что вам самим интересно использовать.